

A Comparative Semantic Retrieval System for Greek Logic and Nyaya/Tarka Sastra using Retrieval-Augmented Generation (RAG)

Developed by

Taticherla Hasini

B.Tech (Computer Science Engineering)

Indian Institute of Information Technology, Design and Manufacturing Kurnool

1. Project Goal

The primary objective of this project is to develop an intelligent educational system that establishes a clear connection between Greek Logic and the Indian school of Nyaya/Tarka Sastra. Although both traditions focus on valid reasoning, inference, evidence, argument construction, debate, and logical fallacies, they differ in their terminology, structure, and methods of analysis. The project will study these traditions and create a systematic mapping between their corresponding concepts and reasoning processes. This mapping will highlight both their similarities and differences, enabling learners to understand the development of logical thought across two distinct intellectual traditions.

The system will support two-way learning by helping students familiar with Greek Logic understand Nyaya/Tarka Sastra and enabling students of Nyaya/Tarka Sastra to learn Greek Logic. It will be implemented as a retrieval-based question-answering system containing organized information, definitions, examples, inference patterns, and comparative explanations. When a user submits a question, the system will retrieve relevant information from its knowledge base and provide a clear, reliable, and context-specific response. Through this approach, the project aims to make comparative logic more accessible while demonstrating the use of artificial intelligence and information retrieval in developing educational tools.

2. Methodology

The system follows a Retrieval-Augmented Generation (RAG) architecture, built through the following stages:

The proposed system follows a Retrieval-Augmented Generation (RAG) architecture divided into three primary operational pipelines:

- **Document Preprocessing Pipeline:** Covers corpus collection, PDF text extraction via PyPDF, text cleaning, paragraph segmentation, and sliding-window text chunking.
- **Semantic Retrieval Pipeline:** Focuses on dense vector embedding generation using `intfloat/multilingual-e5-large-instruct`, batch processing, and vector indexing within a Qdrant database.
- **Retrieval & Generation Pipeline:** Handles user query vectorization, cosine similarity retrieval of the top-k relevant document chunks, context integration into a prompt, and source-grounded response generation using the Qwen2.5-3B-Instruct Large Language Model.

2.1 Corpus Collection

The first stage of the project involved collecting a reliable and diverse set of reference materials related to Greek Logic and Nyaya/Tarka Sastra. The corpus included primary sources such as classical Greek logic texts and traditional Nyaya texts, along with secondary sources such as academic books, research papers, scholarly articles, and comparative studies. These materials provided information about important concepts, inference methods, reasoning structures, terminology, and logical fallacies in both traditions.

The collected documents form the external knowledge base of the Retrieval-Augmented Generation (RAG) system. Instead of depending only on the general knowledge of a Large Language Model (LLM), the system retrieves relevant information from this corpus when answering a user's question. This helps the system generate responses that are more relevant, traceable, and closely connected to the selected source material.

2.2 PDF Text Extraction

Most of the collected reference materials were available in PDF format. However, a PDF is primarily designed for displaying documents and may store text according to page layout rather than logical reading order. Therefore, the PDF files could not be directly used as input for the later stages of the RAG pipeline.

To convert the PDF documents into machine-readable text, the PyPDF library was used. The library reads each PDF page and extracts its available textual content. The extracted text from all pages was then combined to create a digital text representation of each document. PyPDF provides functions such as `PdfReader` and `extract_text()` for reading PDF files and retrieving page-wise text content. This extraction stage converted the original documents into raw text that could be cleaned, segmented, divided into chunks, and later indexed for retrieval.

2.3 Text Cleaning and Normalization

The text obtained directly from PDF files often contained formatting problems caused by the original page layout. Common issues included unnecessary line breaks, broken words, irregular spacing, repeated headers and footers, page numbers, special characters, and other layout-related artifacts. If this unprocessed text were used directly, it could reduce the quality of document retrieval and produce incomplete or confusing answers.

To solve this problem, a text-cleaning process was applied to each extracted document. Unnecessary spaces and line breaks were removed, broken words were corrected where possible, and irrelevant page elements were discarded. The text was also normalized to create a consistent format across documents from different sources. After cleaning, the processed content was saved in `.txt` format instead of the original `.pdf` format. The text format made the documents easier to inspect, process, segment, and use in subsequent stages of the RAG pipeline.

2.4 Paragraph Segmentation

After cleaning, the continuous text was divided into separate paragraphs. This step was necessary because the extracted PDF text did not always preserve the original paragraph structure. In some cases, a single paragraph was split across multiple lines, while in other cases, headings, page breaks, or formatting elements were incorrectly joined together.

Paragraph segmentation restored a more logical and readable structure to the documents. Each paragraph was treated as a meaningful unit containing a related explanation, argument, definition, or example. The text was divided using paragraph breaks and other structural indicators such as headings and blank lines. Preserving paragraph boundaries helped ensure that the context of each concept remained intact during the later chunking and retrieval stages.

2.5 Text Chunking

Large books and research documents cannot normally be processed as a single input because LLMs and embedding models have limitations on the amount of text they can handle at one time. In addition, embedding an entire document as one unit would make it difficult to retrieve a precise answer for a specific user query. For example, a question about *nyaya* should retrieve the relevant discussion rather than an entire book on *Nyaya* philosophy.

To address this issue, the segmented documents were divided into smaller and manageable text units called chunks. The chunks were created using a suitable number of paragraphs or a fixed

token/character limit while attempting to preserve the meaning and context of the original content. An appropriate chunk size allows the retrieval system to locate relevant information accurately without losing important connections between related sentences.

Each chunk can later be converted into a vector representation using an embedding model and stored in a vector database. When a user asks a question, the system compares the question with the stored chunk representations and retrieves the most relevant passages. These passages are then provided to the LLM as context for generating the final answer. Thus, chunking forms an essential link between document preparation, semantic retrieval, and accurate question answering in the proposed RAG system.

Category	Document Name	Extracted Chunks	Primary Focus
Nyaya Sastra	Nyaya Sutras (Vol. 1)	1,306	Foundational logic & commentary
Nyaya Sastra	Tarka Sangraha	75	Epistemology & Padarthas
Nyaya Sastra	Research Paper on Nyaya	97	Scholarly secondary research
Greek Logic	Organon (Aristotle)	1,130	Aristotelian logic & categories
Greek Logic	Prior Analytics	241	Formal syllogism theory
Greek Logic	Francis Bacon (Stanford Enc.)	99	Inductive logic methodology
Comparative	Comparative Paper 1 (cal)	12	Synthesized comparative analysis
Comparative	Comparative Paper 2 (ca2)	82	Synthesized comparative analysis
Comparative	Comparative Paper 3 (ca3)	54	Synthesized comparative analysis

Table 1: Represents the Knowledge of Extracted Chunks and Corpus.

Chunk Size and Overlap Constraints

During the text-chunking stage, each chunk was maintained within a range of approximately 800 to 1200 characters, with a controlled overlap between consecutive chunks. The maximum limit of 1200 characters was selected to preserve sufficient context by keeping related information, such as the definition, example, and application of a concept, within the same chunk whenever possible. At the same time, a minimum limit of 800 characters was used to prevent the creation of very short and semantically incomplete fragments. For instance, a fragment containing only a word such as "therefore" would not convey meaningful information independently, so it was merged with the preceding content. Overlap was introduced between adjacent chunks to maintain continuity across chunk boundaries. This ensures that logically connected content, such as the major premise, minor premise, and conclusion of an argument, is not separated in a way that causes the retrieval system to lose essential context. These constraints provide a balance between preserving semantic meaning, supporting accurate retrieval, and complying with the input limitations of embedding models and LLMs.

2.6 Metadata Generation

After the documents were cleaned, segmented, and divided into chunks, relevant metadata was generated and stored along with each chunk. Metadata is additional information that describes the origin and structure of a chunk without changing its actual content. It helps the retrieval system organize the processed corpus, locate relevant information, and trace an answer back to the document from which it was obtained.

For each chunk, the stored metadata includes the Similarity score, Source file name, Chunk ID, Character count and Retrieved Text. The source file name identifies the original book, research paper, or reference document. The chunk ID provides a unique identifier for distinguishing one chunk from another, while the paragraph split information records how the chunk was created from the cleaned document. It may also indicate the paragraphs included in the chunk and, where required, its character or token size.

This metadata improves the transparency and reliability of the Retrieval-Augmented Generation system. When a user submits a question, the system retrieves the most relevant chunk and uses its metadata to identify the corresponding source document and location. As a result, the system can determine which book or research paper contributed to an answer, which chunk was retrieved, and how the text was divided during preprocessing. This information is also useful for debugging, evaluating retrieval quality, removing duplicate content, and providing source references in the final response.

2.7 Dense Embedding Generation

After the text was divided into meaningful chunks, each chunk was converted into a numerical representation using the `intfloat/multilingual-e5-large-instruct` embedding model. This numerical representation is called an embedding. It represents the meaning and context of the text in the form of a vector, allowing the system to compare the semantic similarity between a user’s query and the stored document chunks. The selected model produces a 1024-dimensional embedding for each input text.

For example, a chunk may be represented as a vector such as $[0.45, -0.12, 0.98, 0.37, -0.51, \dots]$. These values are not individual words or keywords; collectively, they capture the underlying meaning of the entire chunk. A dense vector contains meaningful numerical values across most of its dimensions, whereas a sparse representation contains mostly zero values and generally focuses on the presence of specific terms.

This project uses dense embeddings because the system is designed for semantic search rather than simple keyword matching. As a result, it can identify related content even when the wording of the query and the wording of the source text are different. For example, a user asking, "How is a conclusion established from supporting evidence?" may retrieve a passage discussing inference or reasoning, even if the exact words in the question do not appear in the stored chunk. The generated embeddings are stored in a vector database, where they can later be compared with the embedding of a user query to retrieve the most relevant information for the RAG system.

2.8 Batch embedding

Generating embeddings for each text chunk individually can be time-consuming, particularly when the corpus contains a large number of paragraphs or documents. In a sequential approach, each chunk is sent to the embedding model through a separate request, with the first chunk processed before the second, followed by the remaining chunks. Although this method is straightforward to implement, it results in repeated model calls and introduces additional processing overhead. To address this limitation, the implementation uses batch processing with a batch size of 32. In this approach, 32 text chunks are grouped together and submitted to the embedding model simultaneously, allowing the model to process them in a single operation and return 32 corresponding embedding vectors. This process continues until all the chunks in the corpus have been converted into embeddings.

Batch embedding reduces the number of individual model calls and makes more effective use of

available computational resources, including CPU memory and GPU parallelism. Consequently, it decreases the total processing time while preserving the same embedding quality as individual processing. The batch size of 32 provides a practical balance between processing speed and memory consumption, since an excessively large batch could require more memory, whereas a very small batch may not provide significant performance improvement. After the embeddings are generated, each vector is associated with its original text chunk and corresponding metadata, such as document information or chunk identifiers. These vectors and their associated details are then stored in the vector database, enabling efficient semantic search and retrieval.

2.9 Vector Database

All generated embeddings are stored in Qdrant, a dedicated vector database designed for the efficient storage and retrieval of high-dimensional vector representations. Qdrant was selected because manually searching through a large collection of vectors would be inefficient and time-consuming. For example, comparing a query vector individually with 3,096 stored vectors would require a large number of similarity calculations and could significantly increase retrieval time. Instead, Qdrant uses optimized nearest-neighbour search techniques to quickly identify the vectors that are most similar to a given query vector, thereby improving the speed and efficiency of semantic search.

During the upload process, each generated embedding is represented as a `PointStruct`. This structure contains a unique identifier, the embedding vector, and a payload containing additional information about the original text chunk, such as its content, document details, or metadata. The `client.upsert()` function is then used to insert these points into the `other_logic_corpus` collection in Qdrant. Once the upload process is completed, the collection contains 3,096 semantic vector representations along with their associated metadata. These stored vectors are then ready to be searched and retrieved when a user submits a query, supporting efficient document retrieval and context-based response generation.

2.10 Qdrant-Based Dense Vector Retrieval

This stage evaluates the retrieval component of the proposed system. When a user submits a question, the question is first converted into a numerical vector, known as an embedding, using the same embedding model that was applied to the corpus chunks. Using the same model for both the query and the document chunks ensures that they are represented within the same semantic space, allowing meaningful comparisons to be made. The generated query embedding is then sent to Qdrant, where it is compared with the vectors stored in the database.

Qdrant uses cosine similarity to measure the semantic closeness between the query vector and each stored chunk vector. Based on the similarity scores, it identifies and returns the chunks that are most relevant to the user's question. Unlike a simple keyword-based search, this approach can identify related meanings even when the exact words used in the query do not appear in the stored text. At this stage, the pipeline operates as a complete retrieval system, beginning with the user's question and ending with the retrieval of the most semantically relevant document chunks.

A sample query evaluation run is given below.

User Query: "what is nyaya?"

Search Configuration: Vector Collection = logic_corpus, Top-K = 6, Distance Metric = Cosine Similarity.

Rank	Similarity Score	Source Document File	Chunk ID	Character Count	Extracted Context
1	0.8516	cp/ca1.txt	3	1,200	"Nyaya is one of the six orthodox or astika schools of Hindu philosophy which deals particularly to the science of logic..."
2	0.8278	nyaya/Nyaya-Sutras-Of-Gautama-vol.-1.txt	1305	1,199	"In this study of Nyaya Philosophy as propounded by Gautama and explained by Vatsyayana... the author has examined the empirical foundations..."
3	0.8235	nyaya/Nyaya-Sutras-Of-Gautama-vol.-1.txt	1298	1,200	"It is possible for a Clincher to be urged even where the man has put forward a sound argument..."
4	0.8222	nyaya/Nyaya-Sutras-Of-Gautama-vol.-1.txt	14	1,200	"Professor Garbe considers Nyaya to be the latest of the six orthodox systems..."
5	0.8221	nyaya/Nyaya-Sutras-Of-Gautama-vol.-1.txt	377	1,200	"This reasoning is not very sound; there is no such fixed rule as that the resultant action must bear upon the same object..."
6	0.8207	nyaya/Nyaya-Sutras-Of-Gautama-vol.-1.txt	21	1,200	"The introduction of Yoga in a work on Nyaya is not altogether inexplicable, if we remember that both Yoga and Nyaya..."

Table 2: Semantic Retrieval Execution Results

2.11 Connecting the Large Language Model (LLM)

The final stage transforms the system from a basic search engine into an intelligent question-answering system. After Qdrant retrieves the document chunks that are most relevant to the user's question, these chunks are collected and combined to create a single context-rich prompt. The prompt contains both the user's question and the relevant information retrieved from the document corpus.

The constructed prompt is then provided to the Qwen2.5-3B-Instruct language model. This instruction-tuned model analyzes the user’s question in relation to the retrieved context and generates a clear, natural-language response. Since the model receives relevant document content as part of the prompt, it can provide an answer that is focused on the available information rather than depending only on its general knowledge.

Along with the generated answer, the system also displays the sources used to produce the response. These sources may include the original document name, text chunk, or other associated metadata stored in the vector database. Presenting the sources improves the transparency and reliability of the system, as users can review the information supporting the answer. This completes the retrieval-augmented generation pipeline by combining document retrieval, context construction, language-model processing, and source citation.

3. Tools and Technologies Used

Component	Technology / Method	Description
PDF Processing	PyPDF (text extraction from raw PDF files)	Used to extract raw text content from PDF files for further processing.
Text Processing	Custom text cleaning and paragraph segmentation	Used to remove unnecessary formatting or characters and divide the extracted text into meaningful paragraphs or chunks.
Embedding Model	intfloat/multilingual-e5-large-instruct	Used to generate dense vector representations of the text. Each text chunk is converted into a 1,024-dimensional embedding vector.
Vector Database	Qdrant	Used to store, index, and retrieve the generated embedding vectors efficiently.
Large Language Model	Qwen/Qwen2.5-3B-Instruct	Used to generate natural-language answers based on the user’s query and the relevant text retrieved from the document corpus.
Similarity Metric	Cosine similarity	Used to measure the semantic similarity between the query embedding and the stored document embeddings.

Table 3: Tools and Technologies used

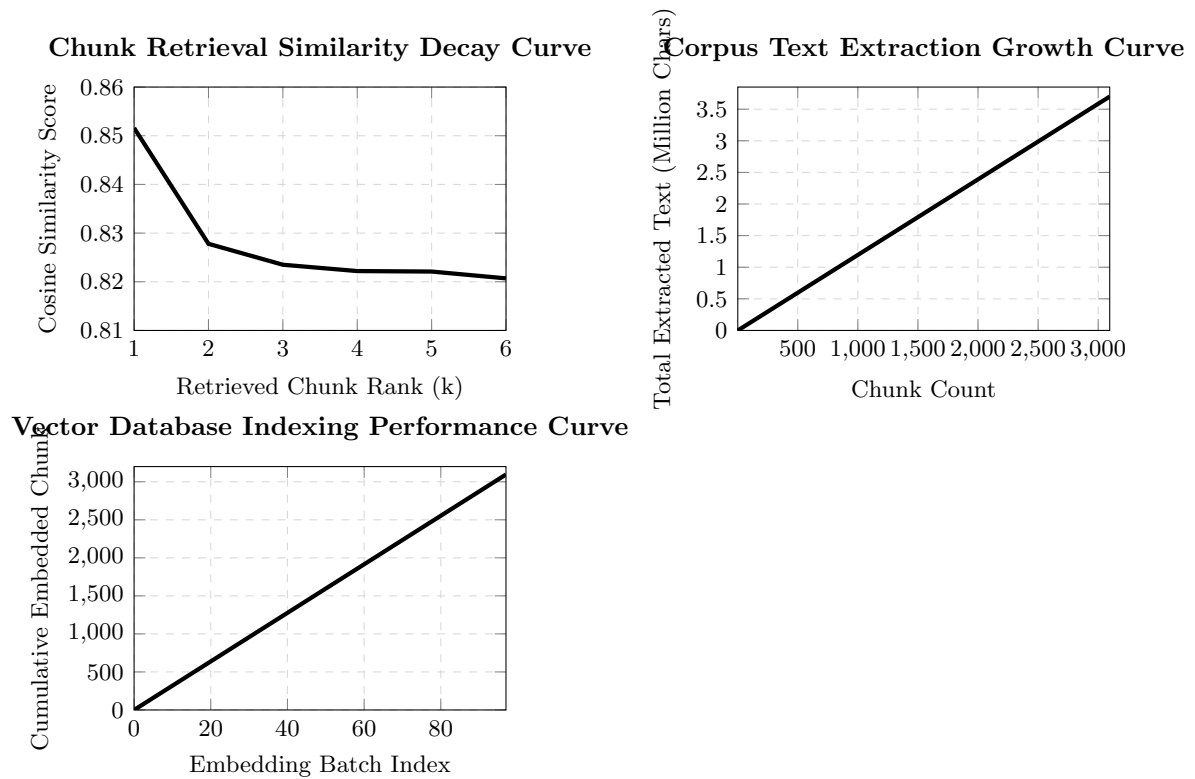
4. Results / Outcome

Upon completion of the proposed pipeline, the vector database contained 3,096 semantic representations generated from the collected corpus of Greek logic and Nyaya/Tarka Sastra materials. Each representation corresponded to a processed text chunk and was stored along with its associated metadata in Qdrant. This organization enabled the system to preserve the relationship between each embedding and its original source, allowing the retrieved information to be traced back to the relevant document or text segment during the question-answering process.

The developed system accepts general questions related to either of the two logical traditions. When a question is submitted, it is converted into an embedding using the same multilingual

embedding model used for the corpus. Qdrant then compares the query embedding with the 3,096 stored embeddings using cosine similarity and retrieves the chunks with the highest semantic relevance. This enables the system to identify conceptually related information even when the wording of the question differs from the wording used in the source documents.

The retrieved chunks are subsequently provided as context to the connected Qwen2.5-3B-Instruct language model. Based on this context, the model generates a clear and relevant answer while also presenting the sources used to support the response. Therefore, the completed implementation functions as a comparative semantic question-answering system capable of working across Greek logic and Nyaya/Tarka Sastra, rather than operating as a conventional keyword-based search tool. It combines document processing, multilingual embedding generation, vector-based retrieval, language-model generation, and source attribution within a unified pipeline.



5. Conclusion

This internship provided hands-on experience in designing and implementing a complete Retrieval-Augmented Generation (RAG) pipeline for connecting two distinct logical traditions—Greek Logic and Nyāya / Tarka Śāstra. The work progressed from collecting reference materials to extracting, cleaning, segmenting, chunking, embedding, storing, retrieving, and finally generating answers using a Large Language Model (LLM).

5.1 What Differentiates the Work from Existing Work

The main contribution of this work is the development of a domain-specific and bidirectional learning framework that focuses specifically on Greek Logic and Nyāya / Tarka Śāstra. Instead of treating the two traditions independently, the system attempts to establish meaningful connections between their concepts, reasoning structures, inference mechanisms, terminology, and logical fallacies.

The major improvements and distinguishing features of the proposed system are:

- **Domain-specific knowledge base:** A dedicated corpus was created using classical texts, academic books, research papers, scholarly articles, and comparative studies related to both logical traditions.
- **Bidirectional learning objective:** The system is designed to help a learner familiar with Greek Logic understand Nyāya / Tarka Śāstra and vice versa.
- **Semantic retrieval instead of keyword-only search:** The use of dense vector embeddings allows the system to retrieve conceptually related passages even when the exact words used in the query are not present in the source text.
- **Multilingual semantic representation:** The `intfloat/multilingual-e5-large-instruct` embedding model was selected to provide stronger semantic representation for multilingual and terminology-rich content.
- **Context-preserving chunking:** Documents were segmented into meaningful paragraphs and chunks rather than treating complete books as single documents. This improves the precision of retrieved information.
- **Traceable responses:** Since answers are generated using retrieved passages from the curated corpus, the responses can remain more closely connected to the selected reference material than answers generated solely from an LLM’s pretrained knowledge.
- **Scalable vector storage:** Qdrant was used as the vector database, allowing the generated embeddings to be efficiently stored and searched for relevant passages.

Improvement Over Conventional Approaches

A conventional LLM-based approach depends mainly on the knowledge encoded during model training. Such an approach may provide general explanations but can produce incomplete, outdated, or unsupported information for specialized philosophical concepts. The proposed RAG approach improves this by introducing an external, curated knowledge base and retrieving relevant source passages before generating an answer.

The project corpus was processed into 3,096 text chunks, which were converted into vector representations and stored in Qdrant for semantic retrieval. This provided a structured foundation for retrieving relevant information instead of passing complete documents to the LLM.

Numerical comparison with older models was not established during the internship because a controlled benchmark using identical datasets, queries, evaluation metrics, and baseline models was not performed. Therefore, specific percentage improvements in accuracy or retrieval performance should not be claimed without experimental evaluation. The current numerical statistics describe the implemented pipeline rather than a direct benchmark against older models.

5.2 Challenges Faced and Their Resolution

Several challenges were encountered during the development of the RAG pipeline.

PDF text extraction: PDF files were designed primarily for document presentation, and their text was not always stored in a logical reading order.

Resolution: PyPDF was used to extract page-wise textual content and combine it into machine-readable text.

Formatting and text-cleaning issues: Extracted text contained unwanted line breaks, irregular spacing, broken words, headers, footers, and page numbers.

Resolution: A preprocessing and normalization stage was implemented to remove irrelevant formatting and produce consistent `.txt` documents.

Loss of paragraph structure: PDF extraction sometimes merged or split paragraphs incorrectly.

Resolution: Paragraph segmentation was performed using paragraph breaks, headings, blank lines, and other structural indicators.

Large document size: Complete books could not be directly processed as a single input for embedding and retrieval.

Resolution: The documents were divided into smaller, meaningful chunks while attempting to preserve their contextual relationships.

Selecting an appropriate embedding model: The system required an embedding model capable of representing philosophical concepts and handling multilingual content.

Resolution: `intfloat/multilingual-e5-large-instruct` was selected to generate dense semantic embeddings for both documents and queries.

Computational limitations: Generating embeddings for thousands of chunks can be computationally expensive, particularly when running without a dedicated GPU.

Resolution: Embedding generation was performed in batches and GPU availability was checked so that CUDA could be used when available.

Vector database management: During development, issues occurred when an existing local Qdrant storage directory was accessed by more than one Qdrant client instance.

Resolution: The Qdrant client and storage configuration were managed to ensure that only one active instance accessed the local database at a time.

Retrieval quality: Retrieving an entire document for every query would introduce unnecessary information and reduce the relevance of the context provided to the LLM.

Resolution: Semantic vector search was implemented so that the most relevant chunks could be retrieved for a particular user query.

5.3 Deliverables

The major deliverables completed during the internship were:

- Curated knowledge corpus containing reference materials related to Greek Logic and Nyāya / Tarka Śāstra.
- PDF-to-text extraction pipeline using the PyPDF library.
- Text-cleaning and normalization pipeline for removing unwanted PDF formatting artifacts.
- Paragraph segmentation module for restoring meaningful document structure.
- Text-chunking pipeline for dividing large documents into retrieval-friendly units.
- 3,096 processed text chunks prepared for semantic indexing.
- Embedding generation pipeline using `intfloat/multilingual-e5-large-instruct`.
- Qdrant vector database containing the generated embeddings for semantic retrieval.
- Semantic query-retrieval module capable of retrieving relevant chunks for user queries.
- RAG pipeline architecture connecting document retrieval with LLM-based answer generation.
- Domain-focused question-answering framework designed to support comparative learning between Greek Logic and Nyāya / Tarka Śāstra.

- Project documentation and internship report describing the methodology, implementation, challenges, and outcomes.

Overall, the internship established a functional foundation for a specialized RAG-based educational system. The work demonstrates that combining a curated philosophical corpus, semantic embeddings, vector retrieval, and LLM-based generation can provide a structured approach for learning and comparing two independent logical traditions. Future improvements can focus on quantitative benchmarking, retrieval evaluation, reranking, improved citation mechanisms, and systematic comparison with baseline and older retrieval or language models.

6. Future Scope

The current project provides a foundation for a domain-specific RAG system for Greek Logic and Nyāya / Tarka Śāstra. However, the limited internship duration and computational resources restricted the size of the knowledge base, extensive experimentation, and quantitative evaluation. The following improvements can be considered in future work.

6.1 Addressing the Knowledge Gap

The current knowledge base contains selected texts related to Greek Logic and Nyāya / Tarka Śāstra. Since both traditions have a much larger body of literature, future work can include more original texts, translations, commentaries, research papers, and comparative studies. This will reduce the knowledge gap and improve the reliability and coverage of the system.

6.2 Improving Accuracy of Results

The accuracy of retrieval and generated answers can be improved by experimenting with different embedding models, chunk sizes, and retrieval techniques. Methods such as reranking, hybrid retrieval, query expansion, and metadata-based retrieval can help retrieve more relevant information. Evaluation can be performed using metrics such as precision, recall, and MRR.

6.3 Processing Time and Computational Improvements

Processing large collections requires considerable time, especially during embedding generation and vector indexing. Future improvements can include GPU acceleration, batch processing, parallel processing, caching, and optimized vector search. These methods can reduce processing time and allow larger datasets to be handled efficiently.

6.4 Expanding the Knowledge Base to Other Texts

The proposed pipeline can be extended to additional philosophical and logical texts. More works from Greek philosophers and Indian philosophical schools, along with modern textbooks and research literature, can be incorporated. This would enable the system to answer a wider range of questions and perform deeper comparisons.

6.5 Development of an Interactive Learning Interface

A user-friendly web interface can be developed to allow students to interact with the system directly. It can provide natural-language question answering, retrieved source passages, references, side-by-side concept comparisons, explanations of logical fallacies, and interactive learning activities.

6.6 Application in Other Contexts

The RAG approach developed in this project can be applied to other domains containing large collections of reliable documents. Possible applications include education, legal studies, historical research, academic research, digital humanities, and institutional knowledge systems. The same architecture can be adapted for domain-specific question answering.

6.7 Future Development Under Time Constraints

Due to the limited internship duration, the main focus was on establishing the complete RAG pipeline rather than conducting large-scale experiments and benchmarking. Future work can proceed by expanding the corpus, optimizing processing time, improving retrieval methods, and performing systematic evaluation.